

SKILL-03

To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality.

To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind.

CODE:-

```
GNU nano 8.7.1 skill-03.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <termios.h>

#define INITIAL_SIZE 32
#define MAX_COMMAND_SIZE 1024

/* Linked list node for command history */
typedef struct HistoryNode {
    char *command;
    struct HistoryNode *next;
} HistoryNode;

HistoryNode *historyHead = NULL;
HistoryNode *historyTail = NULL;

/* Disable terminal canonical mode */
void enableRawMode(struct termios *original)
{
    struct termios raw;

    tcgetattr(STDIN_FILENO, original);
    raw = *original;

    raw.c_lflag &= ~(ICANON | ECHO);
    raw.c_cc[VMIN] = 1;
    raw.c_cc[VTIME] = 0;

    tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw);
}

/* Restore normal terminal mode */
void disableRawMode(struct termios *original)
{
    tcsetattr(STDIN_FILENO, TCSAFLUSH, original);
}

[ Read 439 lines ]
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   ^U Undo       ^_ Set Mark   ^] To Bracket
^X Exit      ^R Read File  ^N Replace    ^U Paste      ^J Justify    ^_ Go To Line  ^R Redo       ^G Copy       ^B Where Was
```

```
GNU nano 8.7.1 skill-03.c
    tcsetattr(STDIN_FILENO, TCSAFLUSH, original);
}

/* Add command to history */
void addHistory(const char *command)
{
    HistoryNode *newNode;

    if (command == NULL || strlen(command) == 0)
        return;

    newNode = (HistoryNode *)malloc(sizeof(HistoryNode));

    if (newNode == NULL) {
        perror("malloc");
        return;
    }

    newNode->command = malloc(strlen(command) + 1);

    if (newNode->command == NULL) {
        perror("malloc");
        free(newNode);
        return;
    }

    strcpy(newNode->command, command);
    newNode->next = NULL;

    if (historyHead == NULL) {
        historyHead = newNode;
        historyTail = newNode;
    } else {
        historyTail->next = newNode;
        historyTail = newNode;
    }
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   ^U Undo       ^_ Set Mark   ^] To Bracket
^X Exit      ^R Read File  ^N Replace    ^U Paste      ^J Justify    ^_ Go To Line  ^R Redo       ^G Copy       ^B Where Was
```

```
srikargLAPTOP-KHLR30TF: ~/
GNU nano 8.7.1 skill-03.c
    historyTail = newNode;
}

/* Count number of history commands */
int historyCount(void)
{
    int count = 0;
    HistoryNode *current = historyHead;

    while (current != NULL) {
        count++;
        current = current->next;
    }

    return count;
}

/* Get command at a particular index */
const char *getHistoryCommand(int index)
{
    int i = 0;
    HistoryNode *current = historyHead;

    while (current != NULL) {
        if (i == index)
            return current->command;

        i++;
        current = current->next;
    }

    return NULL;
}

/* Clear the current line on terminal */
void clearLine(void)
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   ^U Undo       ^_ Set Mark   ^- To Bracket
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line ^B Redo       ^G Copy       ^B Where Was
```

```
srikargLAPTOP-KHLR30TF: ~/
GNU nano 8.7.1 skill-03.c
void clearLine(void)
{
    printf("\n\033[K");
    fflush(stdout);
}

/* Display a command on the terminal */
void displayCommand(const char *command)
{
    clearLine();

    printf("mini-shell> %s", command);

    fflush(stdout);
}

/* Free complete history linked list */
void freeHistory(void)
{
    HistoryNode *current = historyHead;
    HistoryNode *nextNode;

    while (current != NULL) {
        nextNode = current->next;

        free(current->command);
        free(current);

        current = nextNode;
    }

    historyHead = NULL;
    historyTail = NULL;
}

/* Read command dynamically */
char *readCommand(void)
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   ^U Undo       ^_ Set Mark   ^- To Bracket
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line ^B Redo       ^G Copy       ^B Where Was
```

```
GNU nano 8.7.1 skill-03.c
char *readCommand(void)
{
    size_t capacity = INITIAL_SIZE;
    size_t length = 0;

    char *buffer;
    int ch;

    int historyIndex;
    int count;

    buffer = malloc(capacity);

    if (buffer == NULL) {
        perror("malloc");
        return NULL;
    }

    buffer[0] = '\0';

    /*
     * historyIndex:
     * -1 means current/new command
     * 0 means oldest history command
     */
    historyIndex = -1;

    printf("mini-shell> ");
    fflush(stdout);

    while (1) {
        ch = getchar();

        /* Ctrl-D */
        if (ch == EOF || ch == 4) {
            printf("\n");
            free(buffer);
        }
    }
}
```

GNU nano 8.7.1 skill-03.c

Help Exit Write Out Read File Where Is Replace Cut Paste Execute Justify Location Go To Line Undo Redo Set Mark Copy To Bracket Where Was

```
GNU nano 8.7.1 skill-03.c
    if (ch == EOF || ch == 4) {
        printf("\n");
        free(buffer);
        return NULL;
    }

    /* Enter key */
    if (ch == '\n' || ch == '\r') {
        printf("\n");
        buffer[length] = '\0';
        return buffer;
    }

    /* Backspace */
    if (ch == 127 || ch == 8) {
        if (length > 0) {
            length--;
            buffer[length] = '\0';

            printf("\b \b");
            fflush(stdout);
        }
        continue;
    }

    /*
     * Escape sequence
     * Arrow keys normally generate:
     *
     * Up = ESC [ A
     * Down = ESC [ B
     * Right = ESC [ C
     * Left = ESC [ D
     */
    if (ch == 27) {
        int second;
    }
}
```

GNU nano 8.7.1 skill-03.c

Help Exit Write Out Read File Where Is Replace Cut Paste Execute Justify Location Go To Line Undo Redo Set Mark Copy To Bracket Where Was

```
srikargLAPTOP-KHLR30TF: ~/
GNU nano 8.7.1 skill-03.c
if (ch == 27) {
    int second;
    int third;

    second = getchar();

    if (second == '[') {
        third = getchar();

        count = historyCount();

        /* UP ARROW */
        if (third == 'A') {
            if (count > 0) {
                if (historyIndex == -1)
                    historyIndex = count - 1;
                else if (historyIndex > 0)
                    historyIndex--;

                {
                    const char *oldCommand =
                        getHistoryCommand(historyIndex);

                    if (oldCommand != NULL) {
                        size_t oldLength =
                            strlen(oldCommand);

                        if (oldLength + 1 > capacity) {
                            char *temp;

                            capacity = oldLength + 1;

                            temp = realloc(buffer, capacity);

                            if (temp == NULL) {
                                perror("realloc");
                            }
                        }
                    }
                }
            }
        }
    }
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location ^U Undo ^A Set Mark ^] To Bracket
^X Exit ^R Read File ^N Replace ^U Paste ^J Justify ^_ Go To Line ^B Redo ^G Copy ^_ Where Was

```
capacity);

    if (temp == NULL) {
        perror("realloc");
        free(buffer);
        return NULL;
    }

    buffer = temp;
}

strcpy(buffer, oldCommand);
length = oldLength;
displayCommand(buffer);
}
} else {
    /*
     * Moving down from newest command
     * creates a fresh empty command.
     */
    historyIndex = -1;
    length = 0;
    buffer[0] = '\0';

    clearLine();
    printf("mini-shell> ");
    fflush(stdout);
}
}

/* LEFT ARROW */
else if (third == 'D') {
    /* Cursor movement not required */
    fflush(stdout);
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location ^U Undo ^A Set Mark ^] To Bracket
^X Exit ^R Read File ^N Replace ^U Paste ^J Justify ^_ Go To Line ^B Redo ^G Copy ^_ Where Was

```
srikar@LAPTOP-KHLR30TF: ~/   
GNU nano 8.7.1 skill-03.c   
   
while (1) {   
    command = readCommand();   
   
    if (command == NULL)   
        break;   
   
    /*   
     * Store command in linked-list history.   
     */   
    if (strlen(command) > 0)   
        addHistory(command);   
   
    /*   
     * Display what the user entered.   
     */   
    if (strlen(command) > 0) {   
        printf("Command entered: %s\n", command);   
    }   
   
    /*   
     * Free dynamically allocated input buffer.   
     */   
    free(command);   
   
}   
   
/* Restore terminal before exiting */   
disableRawMode(&originalTerminal);   
   
/* Free complete history */   
freeHistory();   
   
printf("Program terminated safely.\n");   
   
return 0;   
}   
   
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   ^U Undo       ^M Set Mark   ^_ To Bracket   
^X Exit      ^R Read File  ^N Replace    ^U Paste      ^O Justify    ^V Go To Line ^B Redo       ^G Copy       ^B Where Was
```

OUTPUT:-

```
venu@LAPTOP-KHLR30TF: ~/   
Processing triggers for man-db (2.13.1-1build1) ...   
Processing triggers for libc-bin (2.43-2ubuntu2.4) ...   
srikar@LAPTOP-KHLR30TF:~/os-lab/skills$ lgrind --version   
valgrind-3.26.0   
srikar@LAPTOP-KHLR30TF:~/os-lab/skills$ ./os-lab/skill   
srikar@LAPTOP-KHLR30TF:~/os-lab/skills$ c -Wall -Wextra -g skill-03.c -o skill-03   
srikar@LAPTOP-KHLR30TF:~/os-lab/skills$ lgrind --leak-check=full --show-leak-kinds=all ./skill-03   
==1986== Memcheck, a memory error detector   
==1986== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.   
==1986== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info   
==1986== Command: ./skill-03   
==1986==   
=====   
Dynamic Command History Program   
=====   
Type commands and press Enter.   
Use UP/DOWN arrows for history.   
Use Backspace to edit.   
Press Ctrl-D to exit.   
=====   
mini-shell> hello   
Command entered: hello   
mini-shell> world   
Command entered: world   
mini-shell>   
Program terminated safely.   
==1986==   
==1986== HEAP SUMMARY:   
==1986==    in use at exit: 0 bytes in 0 blocks   
==1986==   total heap usage: 9 allocs, 9 frees, 2,188 bytes allocated   
==1986==   
==1986== All heap blocks were freed -- no leaks are possible   
==1986==   
==1986== For lists of detected and suppressed errors, rerun with: -s   
==1986== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)   
srikar@LAPTOP-KHLR30TF:~/os-lab/skills$
```